# TECNOLÓGICO NACIONAL DE MÉXICO

# CAMPUS COLIMA

## Práctica 2

Buscador de Cadena o Caracter en un Archivo de Texto

Unidad IV

**Carrera:**

Ingeniería en Sistemas Computacionales

**Materia:**

Lenguajes y Automatas I

**Alumnos**

Alvarez Neri Laura Naomi
Grageda Sandoval Sebastian

**Profesor**

Juan Garcia Virgen

Villa de Álvarez Col, a 17 de abril de 2026

## Introducción

El propósito de este ejercicio es crear un programa en Python que esté diseñado para interactuar con archivos de texto externos. El propósito principal es que el usuario pueda buscar caracteres o cadenas de texto concretos en un archivo local, informando de forma precisa y detallada la frecuencia con la cual se presenta el elemento buscado y su ubicación (número de línea). Se centra en la gestión de archivos, el control de excepciones y el procesamiento de cadenas.

**Desarrollo**

Para el planteamiento del programa, se siguió una lógica secuencial dividida en los siguientes puntos:

1. **Manejo de Archivos:** Se utilizó la función `open()` con un bloque `with` para asegurar el cierre automático del archivo. Se implementó un manejo de excepciones (`try-except`) para evitar que el programa se detenga si el nombre del archivo es incorrecto.
2. **Lectura de Datos:** El contenido se carga en una lista de líneas mediante `readlines()`, lo que facilita la iteración y el rastreo del número de línea.
3. **Lógica de Búsqueda:** Se utiliza el método `.count()` sobre cada línea para obtener la cantidad exacta de coincidencias, acumulándolas en una variable global de conteo.
4. **Presentación de Resultados:** Se emplea una enumeración (`enumerate`) para mostrar al usuario exactamente en qué líneas se encontró el texto y un resumen final con el total de ocurrencias.

Corridas de Prueba

1. **Caso de éxito:** Captura buscando una palabra que sí existe varias veces.

```python
def buscar_en_archivo():
    if encontrado:
        # Se encontró al menos una ocurrencia: se informa el total.
        print(
            f"Resultado: '{busqueda}' se encontró {total_ocurrencias} "
            f"vez/veces en total."
        )
    else:
        # La cadena buscada no apareció en ninguna línea del archivo.
        print(f"Resultado: '{busqueda}' NO se encontró en el archivo.")


# ==================================================================
# PUNTO DE ENTRADA DEL PROGRAMA
# ==================================================================
# Esta condición asegura que buscar_en_archivo() solo se ejecute cuando
# el script se corre directamente (ej. "python buscador.py"), y NO cuando
# este archivo sea importado como módulo desde otro script.
if __name__ == "__main__":
    buscar_en_archivo()
```

```
PS C:\Users\sebas\OneDrive\Escritorio\cosas> & C:/Users/sebas/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/sebas/OneDrive/Escritorio/cosas/BuscarCadenas.py
Ingresa el nombre del archivo de texto: poema.txt
Ingresa la cadena o carácter a buscar: gato

Buscando 'gato' en 'poema.txt'...

  Línea 1: 'El gato duerme en el techo' -> encontrado 1 vez/veces
  Línea 3: 'El gato corre por la casa' -> encontrado 1 vez/veces

Resultado: 'gato' se encontró 2 vez/veces en total.
PS C:\Users\sebas\OneDrive\Escritorio\cosas>
```

2. **Caso de no coincidencia:** Captura buscando una palabra que no está en el texto.

```
PS C:\Users\sebas\OneDrive\Escritorio\cosas> & C:/Users/sebas/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/sebas/OneDrive/Escritorio/cosas/BuscarCadenas.py
Ingresa el nombre del archivo de texto: poema.txt
Ingresa la cadena o carácter a buscar: automatas

Buscando 'automatas' en 'poema.txt'...


Resultado: 'automatas' NO se encontró en el archivo.
PS C:\Users\sebas\OneDrive\Escritorio\cosas>
```

3. **Caso de error:** Captura un fragmento de la palabra y el sistema detecta que no es una cadena y salta una excepción.

```
PS C:\Users\sebas\OneDrive\Escritorio\cosas> & C:/Users/sebas/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/sebas/OneDrive/Escritorio/cosas/BuscarCadenas.py
Ingresa el nombre del archivo de texto: poema.txt
Ingresa la cadena o carácter a buscar: to

Buscando 'to' (palabra exacta) en 'poema.txt'...


Resultado: 'to' NO se encontró en el archivo.
PS C:\Users\sebas\OneDrive\Escritorio\cosas>
```

**Comentarios y Conclusiones**

Comentarios: El código es eficiente para archivos de texto de tamaño pequeño y mediano. El uso de `encoding='utf-8'` es un acierto técnico, ya que permite leer caracteres especiales o acentos sin que el programa falle por errores de codificación.

Conclusiones:

- Aprendizaje Técnico: Se reforzó el uso de estructuras de control y métodos de cadenas como `.rstrip()` y `.count()`.
- Importancia del Manejo de Errores: La implementación de `try-except` es vital, ya que el error de "archivo no encontrado" es uno de los más comunes por parte del usuario final.
- Utilidad: Este tipo de lógica es la base para herramientas de filtrado de logs o análisis de datos en archivos planos, demostrando la versatilidad de Python para la automatización de tareas de oficina o programación.